

Triton: 面向分块神经网络计算的中间语言与编译器

论文精读与全文解读

原论文作者: Philippe Tillet, H. T. Kung, David Cox
精读整理

原论文发表于 MAPL ' 19, 2019年6月22日

Abstract

本文档是对 Philippe Tillet 等人于 2019 年在 MAPL' 19 会议上发表的论文 “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations” 的逐段全文精读。文档按照原论文的章节顺序, 对每一段提供完整翻译(灰色框)、通俗解释(绿色框)、数学推导详解(蓝色框)、关键概念总结(橙色框)和注意事项(红色框), 并嵌入原论文中所有图表并给出详细中文解读。

Contents

1 摘要 (Abstract)	3
2 引言 (Introduction)	3
3 相关工作 (Related Work)	9
4 Triton-C 语言	10
4.1 语法 (Syntax)	10
4.2 语义 (Semantics)	11
4.2.1 Tile 语义	11
4.2.2 广播语义	12
4.3 编程模型	13
5 Triton IR	15
5.1 结构 (Structure)	15
5.1.1 模块 (Modules)	15
5.1.2 函数 (Functions)	15
5.1.3 基本块 (Basic Blocks)	15
5.2 支持 Tile 级数据流分析	16
5.2.1 类型	16
5.2.2 指令	16
5.3 支持 Tile 级控制流分析	18

6 Triton-JIT 编译器	19
6.1 机器无关优化遍	19
6.1.1 预取 (Pre-Fetching)	19
6.1.2 Tile 级窥孔优化	20
6.2 机器相关优化遍	20
6.2.1 层次化分块 (Hierarchical Tiling)	20
6.2.2 内存合并 (Memory Coalescing)	21
6.2.3 共享内存分配 (Shared Memory Allocation)	22
6.2.4 共享内存同步 (Shared Memory Synchronization)	23
6.3 自动调优器 (Auto-tuner)	24
7 数值实验	25
7.1 矩阵乘法	25
7.2 卷积	27
7.2.1 稠密卷积	29
7.2.2 移位卷积 (Shift Convolutions)	30
8 结论	32
9 致谢	33
附录：参考文献一览表	34

1 摘要 (Abstract)

原文完整翻译

深度学习领域中新研究思想的验证与部署，往往受限于某些基本原语 (primitive) 的高效计算内核 (compute kernel) 的可用性。尤其是那些无法利用现有厂商库 (如 cuBLAS、cuDNN) 的操作，除非由专家编写定制实现，否则面临设备利用率低下的风险——而这通常以牺牲可移植性为代价。因此，开发新的编程抽象 (programming abstraction)，以最小的性能代价来指定自定义深度学习工作负载，已变得至关重要。

我们提出了 Triton，一种以 **tile** (分块) 概念为核心的语言和编译器。所谓 tile，即静态形状的多维子数组。我们的方法围绕以下两点展开：(1) 一种基于 C 的语言和一种基于 LLVM 的中间表示 (IR)，用于以参数化 tile 变量上的操作来表达张量程序；(2) 一组新颖的 tile 级别优化遍 (pass)，将这些程序编译为高效的 GPU 代码。我们展示了 Triton 如何用于构建与手工调优的厂商库 (cuBLAS / cuDNN) 性能相当的矩阵乘法和卷积内核的可移植实现，或者用于高效实现移位卷积 (shift convolution) 等近期研究思想。

通俗解释

这篇论文提出了一个叫 **Triton** 的新工具。问题背景是：深度学习需要在 GPU 上做大量计算，NVIDIA 提供了 cuBLAS (矩阵运算库) 和 cuDNN (深度学习库) 这样的官方库，但它们只支持有限的几种标准操作。如果研究者想尝试一种新型计算 (比如新的卷积方式)，就必须自己手写 GPU 代码，这非常困难且不可移植。

Triton 的核心创新是引入了「tile (分块)」这个概念——把大矩阵/张量切成固定形状的小块来处理。程序员只需要描述对这些小块做什么操作，Triton 的编译器会自动生成高效的 GPU 代码。最终效果是：用 Triton 写的代码性能可以媲美 NVIDIA 官方库，但编写难度大大降低。

关键概念/总结

- **问题**：深度学习的新操作缺乏高效 GPU 实现，现有厂商库覆盖范围有限。
- **方案**：Triton——基于「tile (分块)」概念的语言 + 编译器。
- **核心组件**：(1) Triton-C 语言 (类 C 前端)；(2) Triton-IR (基于 LLVM 的中间表示)；(3) Triton-JIT (即时编译器 + 自动调优器)。
- **效果**：矩阵乘法性能媲美 cuBLAS，卷积性能媲美 cuDNN，且支持移位卷积等新操作。

2 引言 (Introduction)

原文完整翻译

深度神经网络 (DNN) 近年来的复兴在很大程度上是由可编程并行计算设备的广泛可用性所推动的 **【[24] Krizhevsky et al. 2012, ImageNet 分类论文——这里被引用是因为 AlexNet 的成功标志着 GPU 加速深度学习时代的开始】**。具体而言，众核架构 (如 GPU) 性能的持续提升发挥了根本性作用，使研究者和工程师能够使用越来越多的数据来探索日益增长的各种大型模型。这一努力得到了一系列厂商库 (cuBLAS、cuDNN) 的支持，这些库旨在尽快将最新的硬件创新带给实践者。不幸的是，这些库仅支持一组受限的张量操作，将新型原语的实现留给了专家 **【[13] Diamos et al. 2016, Persistent RNNs**

——自定义 GPU 内核的例子】 【[17] Gray et al. 2017, 块稀疏 GPU 内核——另一个需要专家手写的例子】 【[25] Lavin & Gray 2016, Winograd 快速卷积——专家优化卷积的例子】。

这一观察导致了各种面向 DNN 的领域特定语言 (DSL) 的发展, 基于多面体机器 (如 Tensor Comprehensions 【[43] Vasilache et al. 2018, TC——基于多面体模型的 DNN 编译器】) 和/或循环合成技术 (如 Halide 【[37] Ragan-Kelley et al. 2013, Halide——用于图像处理的语言和编译器】, TVM 【[10] Chen et al. 2018, TVM——端到端深度学习优化栈】和 PlaidML 【[22] Intel AI 2018, PlaidML——Intel 的开源深度学习编译器】)。但尽管这些系统在某些类别的问题 (如深度可分离卷积, 例如 MobileNet 【[20] Howard et al. 2017, MobileNets——轻量级 CNN 架构】) 上通常表现良好, 它们在实践中往往比厂商库慢得多 (参见图 1), 并且缺乏实现结构化稀疏模式所需的表达力 【[28] McDanel et al. 2017, 嵌入式二值化神经网络】 【[31] Narang et al. 2017, 块稀疏 RNN】 【[47] Wu et al. 2017, Shift 操作——零 FLOP 的空间卷积替代方案】, 这些模式无法直接使用嵌套循环中的仿射数组索引来指定。

通俗解释

这段话是在给读者交代背景:

为什么需要 GPU? ——2012 年 AlexNet 的成功证明了 GPU 对深度学习的巨大价值。GPU 有成千上万个核心, 适合做大量并行计算。

厂商库的局限——NVIDIA 提供了 cuBLAS (矩阵乘法等线性代数) 和 cuDNN (卷积等 DNN 操作)。但它们只覆盖「标准操作」。如果你发明了一种新型卷积或新型注意力机制, 这些库帮不了你, 你得自己写 CUDA 代码。

已有的 DSL 方案——学术界开发了一些编译器框架 (TC、Halide、TVM、PlaidML), 试图自动生成高效 GPU 代码。但实际测试表明它们往往比 cuBLAS 慢很多 (如图 1 所示), 而且无法表达某些非规则的访问模式 (如结构化稀疏)。

类比: 这就像你去餐厅点菜, 菜单 (cuBLAS/cuDNN) 上只有固定的几道菜。如果你想吃菜单上没有的菜, 要么自己做 (手写 CUDA), 要么用「自动烹饪机」(DSL 编译器), 但自动烹饪机做出来的菜味道 (性能) 往往不如大厨手工做的。

well for certain classes of problems such as depthwise-separable convolutions (e.g., MobileNet [20]), they are often much slower than vendor libraries in practice (see, e.g., Figure 1), and lack the expressivity necessary to implement structured sparsity patterns [28, 31, 47] that cannot be directly specified using affine array indices in nested loops.

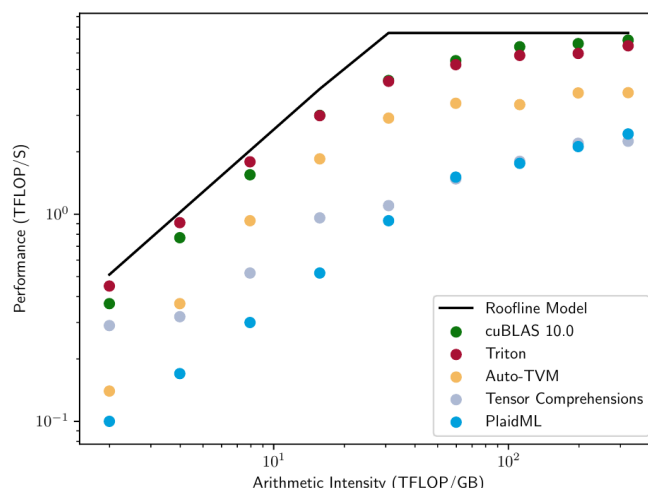


Figure 1. Performance of various implementations of $C = AB^T$ vs. Roofline [46] Model (NVIDIA GeForce GTX1070), where $A \in \mathbb{R}^{1760 \times 1760}$ and $B \in \mathbb{R}^{N \times 1760}$ as N modulates arithmetic intensity.

Figure 1: 不同实现方案对 $C = AB^T$ 运算的性能对比（Roofline 模型），其中 $A \in \mathbb{R}^{1760 \times 1760}$ ， $B \in \mathbb{R}^{N \times 1760}$ ， N 用于调节算术强度。硬件平台为 NVIDIA GeForce GTX1070。

图 1 详细解读

这是一张 **Roofline 模型图**，是 GPU 性能分析中的经典工具：

横轴：算术强度（Arithmetic Intensity），单位为 TFLOP/GB，表示每从内存搬运 1GB 数据能做多少万亿次浮点运算。值越大说明计算越密集、访存越少。

纵轴：性能（Performance），单位为 TFLOP/S，表示每秒实际完成的浮点运算次数。

Roofline 黑线：设备的理论性能上界——左侧受内存带宽限制（斜线部分），右侧受计算峰值限制（水平部分）。

关键发现：

- **cuBLAS**（红色）和 **Triton**（绿色）几乎贴合 Roofline 线，说明它们能充分利用硬件。
- **Auto-TVM**、**Tensor Comprehensions**、**PlaidML** 都显著低于 Roofline 线，尤其在高算术强度区域差距更大。
- 这张图直接证明了论文的核心论点：现有 DSL 编译器在矩阵乘法上的性能远不如 cuBLAS，而 Triton 能达到相当水平。

原文完整翻译

这些问题通常通过使用微内核（micro-kernel）【[11] Chen et al. 2018, TVM（同[10]）】【[21] Huang & van de Geijn 2016, BLISlab——GEMM 优化沙盒框架】——即手写的 tile 级别内建函数——来解决，但这种方案需要大量人工劳动且缺乏可移植性。尽管最近已经提出了几种高级分块编程抽象【[23] Kerr et al., CUTLASS——CUDA 中的快速线性代数库】【[41] Unat et al. 2016, TiDA——数据局部性管理的高级编程抽象】，底层编译器后端仍然缺乏对 tile 级操作和优化的支持。为此我们提出了 Triton（图 2），一个开源^a的中间语言和编译器，用于指定和编译 tile 程序为高效的 GPU 代码。

^a<http://triton-lang.org>

通俗解释

微内核（micro-kernel）指的是针对特定硬件手写的小块运算代码。比如 CUTLASS 库中就有许多手写的矩阵乘法微内核。问题在于：每换一种 GPU，就得重写一遍。

Triton 的定位是：在编译器层面支持 tile 级别的操作。你不需要手写微内核，编译器会帮你做分块、优化内存访问等脏活累活。

compiler for specifying and compiling tile programs into efficient GPU code.

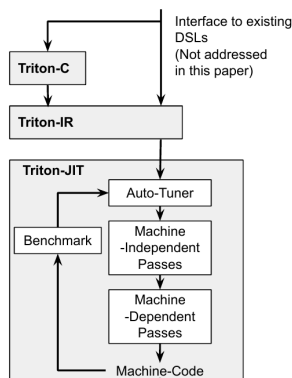


Figure 2: Triton 系统概览

图 2 详细解读

这张图展示了 Triton 的整体架构，从上到下的编译流水线：

Triton-C：用户编写代码的前端语言，语法类似 C/CUDA。也可以作为其他 DSL 的后端接口。

Triton-IR：基于 LLVM 的中间表示。所有优化在这一层进行。

Triton-JIT：即时编译器，包含：

- **Machine-Independent Passes**（机器无关优化）：如预取（prefetching）、窥孔优化。
- **Machine-Dependent Passes**（机器相关优化）：如层次化分块、内存合并、共享内存分配/同步。
- **Auto-Tuner**（自动调优器）：搜索最优的分块参数。
- **Benchmark**（基准测试）：为自动调优器提供性能反馈。

最终输出 **Machine-Code**（机器代码），即可在 GPU 上执行的二进制。

原文完整翻译

本文的主要贡献总结如下：

- **Triton-C**（第 3 节）：一种类 C 语言，用于以参数化 tile 变量表达张量程序。该语言的目的是为现有的 DNN 转译编译器（如 PlaidML, Tensor Comprehensions）和熟悉 CUDA 的程序员提供稳定的接口。代码清单 1 展示了一个简单矩阵乘法任务对应的 Triton-C 源代码。
- **Triton-IR**（第 4 节）：基于 LLVM 的中间表示（IR），为 tile 级别的程序分析、变换和优化提供合适的环境。代码清单 5 展示了修正线性单元（ReLU）函数的 Triton-IR 代码。这里 Triton-IR 程序在解析期间直接从 Triton-C 构建，但未来也可以从嵌入式 DSL 或更高级的 DNN 编译器（如 TVM）自动生成。
- **Triton-JIT**（第 5 节）：即时（JIT）编译器和代码生成后端，用于将 Triton-IR 程序编译为高效的 LLVM 位码。包含：(1) 一组 tile 级别的、机器无关的优化遍，用于独立于任何编译目标来简化输入计算内核；(2) 一组 tile 级别的机器相关优化遍，用于生成高效的 GPU 就绪 LLVM-IR；以及 (3) 一个自动调优器，用于优化与上述优化遍相关的所有元参数。
- **数值实验**（第 6 节）：Triton 的数值评估，展示其能力：(1) 生成与 cuBLAS 性能相当、比其他 DSL 快最多 3 倍的矩阵乘法实现，用于循环和 transformer 神经网络；(2) 在不损失性能的情况下重新实现 cuDNN 的 IMPLICIT_GEMM 密集卷积算法；以及 (3) 为移位卷积（shift-conv）模块等新研究思想创建高效实现。

本文将以现有相关文献的简要分析（第 2 节）开头，以总结和未来工作方向（第 7 节）结束。

通俗解释

这段列出了论文的四大贡献，也正好对应后面的章节结构：

1. **Triton-C**——给程序员用的语言。看起来像 C，但加了 tile 相关的语法。
2. **Triton-IR**——内部中间表示。编译器在这一层做各种优化。
3. **Triton-JIT**——把 IR 编译成真正能在 GPU 上跑的代码，还能自动选最优参数。
4. **实验**——在矩阵乘法和卷积两大核心操作上验证了性能。

代码清单 1: Triton-C 中的 $C = A \times B^T$

Listing 1: Triton-C 中的矩阵乘法 $C = A \times B^T$ 。Triton 特有的关键字以注释标出。

```
1 // Tile shapes are parametric and can be optimized
2 // by compilation backends
3 const tunable int TM = {16, 32, 64, 128}
4 const tunable int TN = {16, 32, 64, 128}
5 const tunable int TK = {8, 16}
6 // C = A * B.T
7 kernel void matmul_nt(float* a, float* b, float* c,
8                       int M, int N, int K)
9 {
```

```

10 // 1D tile of indices
11 int rm[TM] = get_global_range(0);
12 int rn[TN] = get_global_range(1);
13 int rk[TK] = 0 ... TK;
14 // 2D tile of accumulators
15 float C[TM, TN] = 0;
16 // 2D tile of pointers
17 float* pa[TM, TK] = a + rm[:, newaxis] + rk * M;
18 float* pb[TN, TK] = b + rn[:, newaxis] + rk * K;
19 for(int k = K; k >= 0; k -= TK) {
20     bool check_k[TK] = rk < k;
21     bool check_a[TM, TK] = (rm < M)[: , newaxis] && check_k;
22     bool check_b[TN, TK] = (rn < N)[: , newaxis] && check_k;
23     // load tile operands
24     float A[TM, TK] = check_a ? *pa : 0;
25     float B[TN, TK] = check_b ? *pb : 0;
26     // accumulate
27     C += dot(A, trans(B));
28     // update pointers
29     pa = pa + TK*M;
30     pb = pb + TK*N;
31 }
32 // write-back accumulators
33 float* pc[TM, TN] = c + rm[:, newaxis] + rn * M;
34 bool check_c[TM, TN] = (rm < M)[: , newaxis] && (rn < N);
35 @check_c *pc = C;
36 }

```

代码清单 1 逐段解读

这段代码实现了 $C = A \times B^T$ 的矩阵乘法。让我们逐步理解：

第1–5行：定义 tile 尺寸参数。tunable 关键字表示这些值是可调的，自动调优器会尝试不同组合（如 TM=64, TN=64, TK=16）来找到最优配置。

第10–12行：创建索引 tile。get_global_range(0) 返回当前内核实例在第 0 维的全局索引范围，类似 CUDA 中的 blockIdx.x * blockDim.x + threadIdx.x，但返回的是一个向量而非标量。

第14行：创建一个全零的累加器 tile C ，大小为 $TM \times TN$ 。

第16–17行：创建指向输入矩阵 A 和 B 的指针 tile。newaxis 用于广播——把一维索引扩展为二维指针数组。

第18–27行：主循环——沿 K 维遍历，每次处理 TK 个元素。包含边界检查（check_a, check_b）以避免越界访问，越界时填 0。核心操作 dot(A, trans(B)) 执行 tile 级矩阵乘法并累加到 C 。

第29–31行：将累加器写回全局内存，同样有边界检查（@check_c 是谓词化写入）。

注意事项

关于 tunable 关键字：这在 Triton-C 中存在，但在 Triton-IR 中不存在。参数化的值在 IR 层面已经被自动调优器确定为具体值。这意味着 Triton-IR 程序是「具体化」后的，形状不再可变。

3 相关工作 (Related Work)

原文完整翻译

深度学习框架 [\[\[1\] Abadi et al. 2016, TensorFlow——Google 的开源机器学习框架\]](#) [\[\[9\] Chen et al. 2015, MXNet——灵活高效的分布式机器学习库\]](#) [\[\[36\] PyTorch 2016——Facebook 的开源深度学习框架\]](#) 和库的存在对于新型神经网络架构和算法的涌现至关重要。但尽管线性代数编译器在分析性 [\[\[5\] Bao & Ding 2013, 防御性循环分块——共享缓存的分块策略\]](#) [\[\[48\] Zhao et al. 2018, 重新审视数据中心的循环分块\]](#) 和经验性 [\[\[6\] Baskaran et al. 2010, 参数化分块——自动调参的分块方法\]](#) [\[\[30\] Mehta et al. 2016, TurboTiling——利用预取提升分块性能\]](#) 启发式方面取得了进展, 这些软件仍不可避免地依赖于手工优化的子程序 (如 cuBLAS 和 cuDNN)。这导致了各种面向 DNN 的 DSL 和编译器的发展, 通常基于以下三种不同方法之一:

- **张量级 IR:** XLA [\[\[16\] Google 2017, TensorFlow XLA——加速线性代数编译器\]](#) 和 Glow [\[\[38\] Rotem et al. 2018, Glow——用于神经网络的图降低编译器\]](#) 使用模式匹配将张量程序转换为预定义的 LLVM-IR 和 CUDA-C 操作模板 (如张量收缩、逐元素操作等)。
- **多面体模型:** Tensor Comprehensions (TC) [\[\[43\]\]](#) 和 Diesel [\[\[14\] Elango et al. 2018, Diesel——GPU 上线性代数和神经网络计算的 DSL\]](#) 使用多面体模型 [\[\[18\] Griebel et al. 1998, 多面体模型中的代码生成\]](#) 来参数化和自动化一个或多个 DNN 层的编译。
- **循环合成器:** Halide [\[\[37\]\]](#) 和 TVM [\[\[10\]\]](#) 将张量计算转换为循环嵌套, 可以使用用户定义的 (尽管可能是参数化的 [\[\[11\]\]](#)) 调度来手动优化。

相比之下, Triton 依赖于在传统编译管线中添加 tile 级操作和优化。这种方法提供了: (1) 比 XLA 和 Glow 更高的灵活性; (2) 与 TC 和 Diesel 不同, 支持非仿射张量索引; 以及 (3) 自动推断可能的执行调度, 而在 Halide 或 TVM 中需要手动指定。Triton 的好处是以增加编程工作量为代价的——参见代码清单 2 中这些 DSL 对矩阵乘法的实现对比。

通俗解释

这一节对比了深度学习编译器领域的三大流派:

- 1. 张量级 IR (XLA, Glow):** 把整个计算图分解成已知的操作模板, 然后套用预写好的优化代码。优点是简单, 缺点是只能处理已知模式, 灵活性差。
- 2. 多面体模型 (TC, Diesel):** 用数学方法 (多面体) 分析循环嵌套中的数据依赖, 自动生成并行代码。优点是理论上很强大, 缺点是只能处理仿射 (线性) 索引, 无法处理非规则的访问模式。
- 3. 循环合成 (Halide, TVM):** 把计算和调度分离, 用户指定怎么分块、怎么并行。优点是灵活, 缺点是用户需要手动写调度 (schedule), 这本身就很复杂。

Triton 的定位: 直接在编译器的 IR 层面支持 tile 操作。不需要多面体分析, 不需要用户写调度, 但程序员需要写比一行代码 (如 TF 的 `tf.matmul`) 更多的代码。这是一个有意的折中——用稍多的编程量换取更好的性能和灵活性。

代码清单 2: $C = A \times B^T$ 在各 DSL 中的实现

Listing 2: 矩阵乘法在不同 DSL 中的表达

```

1 C = tf.matmul(A, tf.transpose(B))           // TF
2 C[i, j : I, J] = +(A[i, k] * B[j, k]);     // PlaidML
3 C(i, j) +=! A(i, k) * B(j, k)             // TC
4 tvm.sum(A[i, k] * B[j, k], axis=k)         // TVM

```

通俗解释

对比非常鲜明：在 TensorFlow 中，矩阵乘法只要一行调用。而在 Triton-C（代码清单 1）中则需要约 30 行。但 Triton 的优势在于——这 30 行代码的性能可以媲美 cuBLAS（而其他 DSL 自动生成的代码往往差很多），且可以灵活修改来实现新的计算模式。

关键概念/总结

Triton 与三大流派的对比：

方法	代表	优势	劣势
张量级 IR	XLA, Glow	简单	只支持已知模式
多面体模型	TC, Diesel	理论强大	限于仿射索引
循环合成	Halide, TVM	灵活	需手写调度
Tile 级操作	Triton	自动优化 + 灵活	编程量稍多

4 Triton-C 语言

原文完整翻译

Triton-C 的目的是为现有（和未来的）DNN 转译编译器，以及熟悉低级 GPU 编程的程序员提供稳定的前端。在本节中，我们描述 Triton-C 的类 CUDA 语法（第 3.1 节）、类 Numpy 的语义（第 3.2 节）以及其“单程序多数据”（SPMD）编程模型（第 3.3 节）。

通俗解释

Triton-C 设计时考虑了两类用户：一是熟悉 CUDA C 的 GPU 程序员（语法相似），二是习惯 NumPy 的 Python 开发者（语义相似，如广播规则）。下面分三个方面介绍这门语言。

4.1 语法 (Syntax)

原文完整翻译

Triton-C 的语法基于 ANSI C（更具体地说是 CUDA-C），但进行了修改和扩展（参见代码清单 3）以适应接下来两个小节描述的语义和编程模型。这些变化分为以下几类：

Tile 声明：我们添加了特殊语法来声明多维数组（如 `int tile[16, 16]`），以强调其与 ANSI C 中嵌套数组（如 `int tile[16][16]`）的语义差异。Tile 形状必须是常量，但也可以使用 `tunable` 关键字使其参数化。一维整数 tile 可以使用省略号初始化（如 `int range[8] = 0 ... 8`）。

内建函数：虽然常见的 C 语法保留用于逐元素数组操作（`+`, `-`, `&&`, `*` 等），但添加了各种内建函数（`dot`, `trans`, `get_global_range`）以支持 tile 语义（第 3.2.1 节）和 SPMD 编程模型。

广播： N 维 tile 可以使用 `newaxis` 关键字和通常的切片语法沿任何特定轴进行广播（如 `int broadcast[8, 8] = range[:, newaxis]` 用于堆叠列）。注意，通过切片获取标量或子数组在其他情况下是被禁止的。

谓词化： tile 操作中的基本控制流（第 4.3 节）通过使用 `@` 前缀的谓词化语句来实现。

通俗解释

Triton-C 语法的四个关键扩展：

1. **Tile 声明**——`int a[16, 16]` 声明一个 16×16 的整数 tile。注意用逗号分隔维度（而不是 C 语言的 `[] []`），强调这是一个整体操作的多维块，而不是嵌套的一维数组。
2. **内建函数**——`dot(A, B)` 做 tile 级矩阵乘法，`trans(A)` 做转置。这些操作会被编译器映射到 GPU 的专用硬件指令。
3. **广播（Broadcasting）**——借鉴了 NumPy 的规则。比如一个 `[8]` 的向量加上 `[:, newaxis]` 后变成 `[8, 1]`，再与 `[1, 8]` 的向量相加时自动扩展为 `[8, 8]` 的矩阵。
4. **谓词化**——`@mask *ptr = val` 表示“只在 `mask` 为真的元素位置执行写入”。这是因为 tile 中不同元素可能有不同的条件（比如边界检查），但 tile 操作是整体执行的，不能用普通 `if-else`。

代码清单 3：Triton-C 的语法扩展

Listing 3: Triton-C 语法扩展。蓝色部分假设为已有的 C 语法构造。

```
1 // Broadcasting semantics
2 slice : ':' | 'newaxis'
3 slice_list : slice | slice_list ',' slice
4 slice_expr : postfix_expr | expr '[' slice_list ']'
5 // Range initialization
6 constant_range : expr '...' expr
7 // Intrinsic
8 global_range : 'get_global_range' '(' constant ')'
9 dot : 'dot' '(' expr ',' expr ')'
10 trans : 'trans' '(' expr ',' expr ')'
11 intrinsic_expr : global_range | dot | trans
12 // Predication
13 predicate_expr : '@' expr
14 // Tile extensions for abstract declarators
15 abstract_decl : abstract_decl | '[' constant_list ']'
16 // Extensions of C expressions
17 expr : expr | constant_range | slice_expr | intrinsic_expr
18 // Extensions of C specifiers
19 storage_spec : storage_spec | 'kernel'
20 type_spec : type_spec | 'tunable'
21 // Extensions of C statements
22 statement : statement | predicate_expr statement
```

4.2 语义（Semantics）

4.2.1 Tile 语义

原文完整翻译

Triton-C 中内建 tile 类型和操作（即 tile 语义）的存在带来两大好处。首先，它通过隐藏与 tile 内存合并 [【\[12\] Davidson & Jinturkar 1994, 内存访问合并——消除冗余内存访问的技术】](#)、缓存管理 [【\[32\] Nath et al. 2011, 加速稠密线性代数的 GPU 内核】](#) 和专用硬件利用 [【\[27\] Martineau et al. 2017, NVIDIA V100 GPU 和 Tensor Core 的基准测](#)

试】 相关的重要性能细节，简化了张量程序的结构。其次，它为编译器自动执行这些优化打开了大门，如第 5 节所讨论的。

通俗解释

Tile 语义的核心思想是「把底层优化细节隐藏起来」。当你写 `dot(A, B)` 时，你不需要关心：

- 内存是否被合并访问 (coalesced access)
- 数据是否被缓存在共享内存中
- 是否利用了 Tensor Core 等专用硬件

这些全部由编译器自动处理。你只需要描述「我要做什么运算」，而不是「我要怎么做」。

4.2.2 广播语义

原文完整翻译

Triton-C 中的 tile 是强类型的，某些指令静态地要求其操作数满足严格的形状约束。例如，标量不能直接加到数组上，除非先适当地进行广播。广播语义 [【\[35\] Oliphant 2006, NumPy—Python 数值计算库】](#) 提供了一组规则来隐式执行这些转换（参见代码清单 4 中的示例）：

1. **填充 (Padding)**：最短操作数的形状在左侧用 1 填充，直到两个操作数具有相同的维度数。
2. **广播 (Broadcasting)**：两个操作数的内容按需复制尽可能多次，直到它们的形状相同；如果无法做到则发出错误。

通俗解释

广播规则与 NumPy 完全一致。举个具体例子：

`int a[16]` 形状为 `[16]`，`int b[32, 16]` 形状为 `[32, 16]`。

执行 `a + b` 时：

1. **填充**：`a` 的形状 `[16]` 左填充为 `[1, 16]`。
2. **广播**：`[1, 16]` 的第 0 维复制 32 次变为 `[32, 16]`。
3. 现在两者形状都是 `[32, 16]`，可以逐元素相加。

代码清单 4：广播语义实例

Listing 4: 广播语义实例

```
1 int a[16], b[32, 16], c[16, 1];
2 // a is first reshaped to [1, 16]
3 // and then broadcast to [32, 16]
4 int x_1[32, 16] = a[newaxis, :] + b;
5 // Same as above but implicitly
6 int x_2[32, 16] = a + b;
```

```
7 // a is first reshaped to [1, 16]
8 // a is broadcast to [16, 16]
9 // c is broadcast to [16, 16]
10 int y[16, 16] = a + c;
```

4.3 编程模型

原文完整翻译

CUDA [【\[33\] Nickolls et al. 2008, CUDA——可扩展的并行编程】](#) 代码在 GPU 上的执行由 SPMD [【\[4\] Auguin & Larbey, Opsila——SIMD 处理器架构】](#) 编程模型支持，其中每个内核与所谓的启动网格中的一个可标识的线程块关联。Triton 编程模型类似，但每个内核是单线程的——尽管会自动并行化——并与一组全局范围（global range）关联，这些范围因实例而异（参见图 3）。这种方法使得内核更简单，因为其中不存在 CUDA 式的并发原语（共享内存同步、线程间通信等）。

内核关联的全局范围可以使用 `get_global_range(axis)` 内建函数查询，以创建如代码清单 1 所示的指针 tile。

通俗解释

CUDA 编程模型 vs Triton 编程模型——这是一个关键区别：

CUDA：程序员需要管理线程块（Block）中的多个线程（Thread）。每个线程是一个独立执行单元，程序员需要处理线程间同步（`__syncthreads()`）、共享内存的读写协调等。

Triton：程序员写的是「单线程」程序，但操作的是整个 tile（数据块）。编译器自动决定如何把一个 tile 的操作分配给多个线程并行执行。程序员完全不需要关心线程管理。

类比：CUDA 就像指挥一个乐队——你要告诉每个乐手（线程）什么时候演奏什么音符，还要确保他们同步。Triton 则像写总谱——你只写出最终要演奏的音乐（tile 操作），乐团指挥（编译器）自动分配演奏任务。

communication, etc.) are inexistent.

The global ranges associated with a kernel can be queried using the `get_global_range(axis)` built-in function in order to create e.g., tiles of pointers as shown in Listing 1.

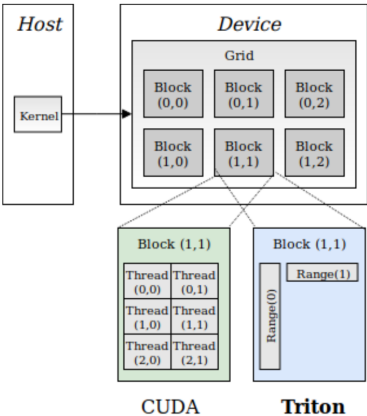


Figure 3. Difference between the CUDA and the Triton programming model

4 The Triton IR

Triton-IR is an LLVM-based Intermediate Representation (IR) whose purpose is to provide an environment suitable for

Figure 3: CUDA 编程模型与 Triton 编程模型的区别

图 3 详细解读

左图（CUDA）：

- Host（CPU）启动 Kernel，在 Device（GPU）上创建一个 Grid（网格）。
- Grid 包含多个 Block（如 Block(0,0) 到 Block(2,1)）。
- 每个 Block 包含多个 Thread（如 Block(1,1) 中有 Thread(0,0) 到 Thread(2,1)）。
- 程序员需要为**每个线程**编写逻辑。

右图（Triton）：

- 同样有 Grid 和 Block 的概念。
- 但 Block(1,1) 中只有 Range(0) 和 Range(1) 两个全局范围。
- 程序员为**每个 tile 实例**编写逻辑，内部并行化由编译器完成。
- 没有线程级别的概念，大大简化了编程。

5 Triton IR

原文完整翻译

Triton-IR 是一种基于 LLVM 的中间表示 (IR)，其目的是为 tile 级程序分析、变换和优化提供合适的环境。在本工作中，Triton-IR 程序在解析期间直接从 Triton-C 构建，尽管未来也可以直接从更高级的 DSL 生成。

Triton-IR 和 LLVM-IR 程序共享相同的高级结构 (第 4.1 节回顾)，但前者还包含一些对 tile 级数据流 (第 4.2 节) 和控制流 (第 4.3 节) 分析所必需的扩展。这些新扩展对于执行第 5 节所述的优化以及安全访问任意形状的张量 (如第 6 节所示) 至关重要。

通俗解释

Triton-IR 是 Triton 系统的「内部语言」。就像 LLVM 用 LLVM-IR 作为前端语言和后端代码生成之间的桥梁一样，Triton-IR 在 Triton-C 和最终 GPU 代码之间充当同样的角色。

关键创新是：在 LLVM-IR 的基础上添加了 tile 级别的扩展。LLVM-IR 只理解标量和向量操作，而 Triton-IR 理解多维 tile 操作，这使得编译器能做更高级的优化。

5.1 结构 (Structure)

5.1.1 模块 (Modules)

原文完整翻译

在最高层，Triton-IR 程序由一个或多个称为模块 (module) 的基本编译单元组成。这些模块彼此独立编译，最终由链接器聚合，链接器的作用是解析前向声明并适当合并全局定义。

每个模块本身由函数、全局变量、常量和杂项符号 (如元数据、函数属性) 组成。

5.1.2 函数 (Functions)

原文完整翻译

Triton-IR 函数定义由返回类型、名称和可能为空的参数列表组成。可以添加额外的可见性、对齐和链接说明符。函数属性 (如内联提示) 和参数属性 (如只读、别名提示) 也可以指定，允许编译器后端执行更积极的优化——例如更好地利用只读内存缓存。

此头部之后是一个由基本块列表组成的函数体，这些基本块的相互依赖关系构成了函数的控制流图 (CFG)。

5.1.3 基本块 (Basic Blocks)

原文完整翻译

基本块按定义是直线代码序列，只能在末尾包含所谓的终结指令 (即分支、返回)。

Triton-IR 使用静态单赋值 (SSA) 形式，这意味着每个基本块中的每个变量必须 (1) 只被赋值一次，且 (2) 在使用前被定义。这样，每个基本块隐式定义了一个数据流图 (DFG)，其不同路径对应于程序 SSA 表示中的使用-定义链。这种形式可以直接从抽象语法树 (AST) 创建，如 [【\[7\] Braun et al. 2013, SSA 形式的简单高效构造方法】](#) 所述。

通俗解释

这三个小节描述了 Triton-IR 从 LLVM-IR 继承的标准结构：

模块 → **函数** → **基本块**，这是编译器 IR 的经典三层架构。

SSA 形式是现代编译器 IR 的标准形式。它的规则是每个变量只赋值一次。比如在普通代码中 $x = 1; x = 2;$ ，在 SSA 中会变成 $x1 = 1; x2 = 2;$ 。这样做的好处是简化了数据流分析——通过跟踪变量名就能知道值从哪里来。

基本块是没有分支的直线代码段。分支 (if/else/for) 只出现在基本块的末尾，用来跳转到其他基本块。

5.2 支持 Tile 级数据流分析

5.2.1 类型

原文完整翻译

多维 tile 是 Triton-IR 中数据流分析的核心，可以使用类似于 LLVM-IR 中向量声明的语法来声明。例如，`i32<8, 8>` 是对应于 8×8 的 32 位整数 tile 的类型。注意 Triton-IR 中没有 `tunable` 关键字，因此参数化形状值必须在生成程序之前解析。在我们的情况下，这由 Triton-JIT 的自动调优器（第 5.3 节）完成。

5.2.2 指令

原文完整翻译

Triton-IR 引入了一组重新分块指令 (retiling instructions)，其目的是支持第 3.2.2 节描述的广播语义：

- `reshape` 指令使用其输入参数的数据创建指定形状的 tile。这对于将变量重新解释为更高维数组（通过在其输入形状上填充 1 来准备隐式或显式广播）特别有用。
- `broadcast` 指令通过沿大小为 1 的维度按需复制其输入参数来创建指定形状的 tile——如图 4 所示。

通常的标量指令 (`cmp`, `getelementptr`, `add`, `load`...) 被保留并扩展为表示对 tile 操作数的逐元素操作。最后，Triton-IR 还暴露了专门的算术指令用于转置 (`trans`) 和矩阵乘法 (`dot`)。

- The `reshape` instruction creates a tile of the specified shapes using data from its input argument. This is particularly useful to re-interpret variables as higher-dimensional arrays by padding their input shapes with ones in preparation of implicit or explicit broadcasting.
- The `broadcast` instruction creates a tile of the specified shapes by replicating its input argument as many times as necessary along dimensions of size 1 – as shown in Figure 4.

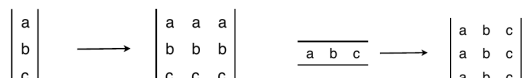


Figure 4: `broadcast <3,3>` 指令的示意图

图 4 详细解读

这张图展示了 broadcast 指令的两种情况：

(a) 3×1 输入：列向量 $[a, b, c]^T$ 被广播为 3×3 矩阵——每一行复制为三列。结果为：

$$\begin{bmatrix} a & a & a \\ b & b & b \\ c & c & c \end{bmatrix}$$

(b) 1×3 输入：行向量 $[a, b, c]$ 被广播为 3×3 矩阵——每一列复制为三行。结果为：

$$\begin{bmatrix} a & b & c \\ a & b & c \\ a & b & c \end{bmatrix}$$

这与 NumPy 的广播行为完全一致，是 Triton 实现高效索引计算的基础。

代码清单 5: Triton-IR 中的 ReLU 函数

Listing 5: $A = \max(A, 0)$ 在 Triton-IR 中的表达。注意 tile 形状在此不再是参数化的，由 Triton-JIT 实例化。

```

1  define kernel void @relu(float* %A, i32 %M, i32 %N) {
2  prologue:
3      %rm = call i32<8> @get_global_range(0);
4      %rn = call i32<8> @get_global_range(1);
5      ; broadcast shapes
6      %1 = reshape i32<8, 8> %M;
7      %M0 = broadcast i32<8, 8> %1;
8      %2 = reshape i32<8, 8> %N;
9      %N0 = broadcast i32<8, 8> %2;
10     ; broadcast global ranges
11     %3 = reshape i32<8, 1> %rm;
12     %rm_bc = broadcast i32<8, 8> %3;
13     %4 = reshape i32<1, 8> %rn;
14     %rn_bc = broadcast i32<8, 8> %4;
15     ; compute mask
16     %pm = icmp slt %rm_bc, %M0;
17     %pn = icmp slt %rn_bc, %N0;
18     %msk = and %pm, %pn;
19     ; compute pointer
20     %A0 = splat float<8, 8> %A;
21     %5 = getelementptr %A0, %rm_bc;
22     %6 = mul %rn_bc, %M0;
23     %pa = getelementptr %5, %6;
24     ; compute result
25     %a = load %pa;
26     %_0 = splat float<8, 8> 0;
27     %result = max %float %a, %_0;
28     ; write back
29     store fp32<8, 8> %pa, %result
30 }
```

代码清单 5 逐段解读

这段 Triton-IR 代码实现了 ReLU 函数 $A = \max(A, 0)$ ，即把矩阵 A 中所有负数置零。

第3–4行：获取当前内核实例在两个维度的全局范围，各得到一个 8 元素的整数向量。

第6–9行：将标量 M 和 N 广播为 8×8 的 tile，用于后续的边界检查。

第11–14行：将一维的全局范围 reshape 和 broadcast 为 8×8 的二维索引 tile。 rm 变成列方向的索引， rn 变成行方向的索引。

第16–18行：计算掩码——只有 $rm < M$ 且 $rn < N$ 的元素才在有效范围内。

第20–23行：计算指向矩阵 A 中对应元素的指针 `tile`。使用 `getelementptr` 进行指针运算，采用列主序（column-major）的布局（偏移 = $rm + rn \times M$ ）。

第25–27行：从内存加载数据，与 0 取最大值（ReLU），然后写回。

5.3 支持 Tile 级控制流分析

原文完整翻译

Triton-IR 中 tile 级操作的存在引发的一个问题是 tile 内部发散控制流的不可表达性。例如，程序可能需要部分保护 tile 级加载以防止内存访问违规，但这无法通过分支实现，因为 tile 元素不能被单独访问。

我们提议通过使用谓词化 SSA（PSSA）形式 [【\[8\] Carter et al. 1999, 谓词化静态单赋值——在 SSA 中支持谓词执行】](#) 和 ψ -函数 [【\[39\] Stoutchinin & de Ferriere 2001, 用于谓词化的高效 SSA 形式】](#) 来解决这个问题。这需要向 Triton-IR 添加两类指令（参见代码清单 6）：

- `cmp` 指令 [【\[8\]】](#) 类似于通常的比较（`cmp`）指令，但它们返回两个相反的谓词而不是一个。
- `psi` 指令合并来自不同谓词化指令流的指令。

通俗解释

在普通程序中，if-else 很容易处理：条件为真走一条路，为假走另一条。但在 tile 级操作中，一个 tile 的不同元素可能有不同的条件结果——有些元素需要走 if 分支，另一些需要走 else 分支。你不能把整个 tile 拆开单独处理每个元素。

解决方案：谓词化执行。

1. `cmp` 指令同时产生两个掩码：一个表示条件为真的元素（`pt`），一个表示条件为假的元素（`pf`）。
2. 两条分支的操作都执行，但各自带上自己的谓词掩码（`@pt` 或 `@pf`）。
3. 最后用 `psi` 指令把两个分支的结果按掩码合并。

类比：这就像 Excel 的 IF 函数——`=IF(A1>5, A1+1, A1-1)` 会对每个单元格独立判断，而不是对整个列做分支。

代码清单 6：Triton-IR 中的 Tile 级谓词化

Listing 6: Triton-IR 中的谓词化示例

```
1 ;pt[i,j], pf[i,j] = (true, false) if x[i,j] < 5
2 ;pt[i,j], pf[i,j] = (false, true) if x[i,j] >= 5
3 %pt, %pf = cmp slt %x, 5
4 @%pt %x1 = add %y, 1
5 @%pf %x2 = sub %y, 1
6 ; merge values from different predicates
7 %x = psi i32<8,8> [%pt, %x1], [%pf, %x2]
8 %z = mul i32<8,8> %x, 2
```

代码清单 6 逐步解读

假设 x 是一个 8×8 的 tile, y 也是 8×8 的 tile:

第3行: `cmppp slt %x, 5` 将 x 的每个元素与 5 比较, 产生两个掩码: `pt` ($x < 5$ 的元素为真) 和 `pf` ($x \geq 5$ 的元素为真)。

第4行: `@%pt %x1 = add %y, 1` 对 `pt` 为真的元素执行 $y + 1$ 。

第5行: `@%pf %x2 = sub %y, 1` 对 `pf` 为真的元素执行 $y - 1$ 。

第7行: `psi` 合并结果——`pt` 为真的位置取 `x1`, `pf` 为真的位置取 `x2`。

第8行: 最终结果统一乘以 2。

6 Triton-JIT 编译器

原文完整翻译

Triton-JIT 的目标是将 Triton-IR 程序简化并编译为高效的机器代码, 通过一组机器无关的 (第 5.1 节) 和机器相关的 (第 5.2 节) 优化遍, 并由自动调优引擎 (第 5.3 节) 提供支持。

6.1 机器无关优化遍

6.1.1 预取 (Pre-Fetching)

原文完整翻译

循环内的 tile 级内存操作可能是有问题的, 因为它们可能引发严重的延迟, 而在没有足够的独立指令时无法隐藏这些延迟。然而, 可以通过在 Triton-IR 中直接检测循环并在必要时添加适当的预取代码来缓解这个问题 (参见代码清单 7)。

通俗解释

预取是一种经典的延迟隐藏技术。问题在于: GPU 从全局内存 (DRAM) 加载数据到寄存器需要数百个时钟周期。如果在循环中先加载数据、再计算, CPU/GPU 在等待数据时就空闲了。

解决方案: 在循环的第 i 次迭代中, 提前加载第 $i + 1$ 次迭代需要的数据。这样, 当第 i 次的计算完成时, 第 $i + 1$ 次的数据已经准备好了。这就是「双缓冲」或「软件流水线」的思想。

代码清单 7 展示了这个变换: 原本循环中先算指针再加载 (load), 变换后在循环外预加载第一次的数据, 循环体中在计算的同时预取下一次的数据。

代码清单 7: 自动预取

Listing 7: 自动预取变换。左侧为原始代码, 右侧为预取优化后的代码。

1	// Original		// After pre-fetching
2	B0:		B0:
3	%p0 = getelementptr		%p0 = getelementptr %1, %2
4	%1, %2		%x0 = load %p0
5	B1:		B1:
6	%p = phi [%p0,B0],		%p = phi [%p0,B0], [%p1,B1]

7	[%p1,B1]		%x = phi [%x0,B0], [%x1,B1]
8	%x = load %p		; increment pointer
9	; increment pointer		%p1 = getelementptr %p, %3
10	%p1 = getelementptr		; prefetching
11	%p, %3		%x1 = load %p

6.1.2 Tile 级窥孔优化

原文完整翻译

Triton-IR 中 tile 级操作的存在为窥孔 [【\[29\] McKeeman 1965, 窥孔优化——经典的局部代码优化技术】](#) 优化器提供了新机会。例如，可以使用恒等式 $X = (X^T)^T$ 对任意 tile X 简化转置链。我们相信，未来还可以利用与对角 tile 等相关的其他代数性质。

通俗解释

窥孔优化（Peephole Optimization）是看一小段代码就能发现的简单优化机会。比如两次连续转置相当于没有转置（ $X = (X^T)^T$ ），编译器可以直接消除这对操作。

这种优化在标量 IR 中也存在（比如 $x + 0 = x$ ），但 tile 级 IR 提供了新的优化机会——基于线性代数的恒等式。

6.2 机器相关优化遍

原文完整翻译

我们现在介绍一组针对遵循图 5 所示高级模型的机器的优化遍。具体而言，Triton-JIT 执行的优化包括：(1) 层次化分块，(2) 内存合并，(3) 共享内存分配，以及 (4) 共享内存同步。

6.2.1 层次化分块（Hierarchical Tiling）

原文完整翻译

嵌套分块策略（参见图 5）旨在将 tile 分解为 micro-tile（微分块），最终分解为 nano-tile（纳分块），以尽可能紧密地适配机器的计算能力和内存层次。虽然这种技术在自动调优框架 [【\[34\] Nugteren 2017, CLBlast——调优的 OpenCL BLAS 库】](#) [【\[40\] Tillet & Cox 2017, 输入感知的自动调优方法】](#) 中常被使用，但 Triton-IR 的结构使得自动枚举和优化任何可表达程序的有效嵌套分块配置成为可能（且无需多面体机器）。

tiles into micro-tiles and eventually nano-tiles in order to fit a machine's compute capabilities and memory hierarchy as tightly as possible. While this technique is routinely used in auto-tuning frameworks [34, 40], the structure of Triton-IR makes it possible to automatically enumerate and optimize valid nested tiling configurations for any expressible program (and without the need for polyhedral machinery).

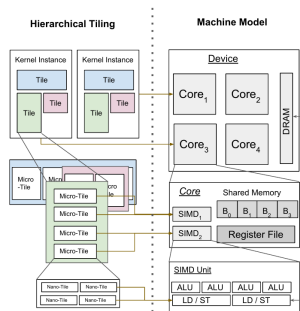


Figure 5. Hierarchical Tiling in the Triton-IR Machine Model

Figure 5: Triton-IR 机器模型中的层次化分块

(e.g., dot) can benefit from temporarily storing their operands in fast shared memory. The purpose of the Shared Memory Allocation pass is to determine when and where a tile should be stashed to this space. This can be done, as illustrated in Figure 7, by first calculating the *live range* of each variable of interest, and then using the linear-time storage allocation algorithm proposed in [15].

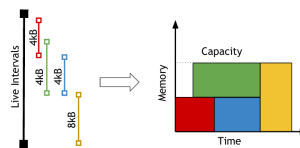


Figure 7. Shared Memory Allocation

5.2.4 Shared Memory Synchronization

Reads from and write to shared memory are asynchronous in our machine model. The goal of the Shared Memory Synchronization pass automatically inserts *barriers* in the generated GPU source code so as to preserve program correctness. This is done by detecting read-after-writes (RAW) and write-after-reads (WAR) dependencies between data flows.

图 5 详细解读

这张图展示了 Triton 的核心优化策略——层次化分块，分为左右两部分：

左侧——层次化分块示意：

- 最外层：每个 **Kernel Instance**（内核实例）处理一个大的 **Tile**。
- 中间层：每个 Tile 被分成多个 **Micro-Tile**（微分块），由 GPU 的不同核心处理。
- 最内层：每个 Micro-Tile 进一步分为 **Nano-Tile**（纳分块），对应 SIMD 单元的处理粒度。

右侧——机器模型：

- **Device**（设备）：包含多个 Core（核心，对应 GPU 的 SM）。
- 每个 Core 有：**Shared Memory**（共享内存）和多个 **SIMD 单元**。
- 每个 SIMD 单元有：**Register File**（寄存器堆）、**ALU**（算术逻辑单元）和 **LD/ST**（加载/存储单元）。
- 外部是 **DRAM**（全局内存）。

对应关系： Tile ↔ 内核实例，Micro-Tile ↔ SIMD 单元（共享内存层），Nano-Tile ↔ 寄存器（寄存器层）。

6.2.2 内存合并（Memory Coalescing）

原文完整翻译

当相邻线程同时访问相近的内存位置时，内存访问被称为是合并的。这很重要，因为内存通常是从 DRAM 以大块形式检索的。

因为 Triton-IR 程序是单线程的并且自动并行化，我们的编译器后端能够在每个 micro-tile 内部对线程进行排序，以尽可能避免非合并内存访问。这种策略减少了加载 tile 列所需的内存事务数量（参见图 6）。

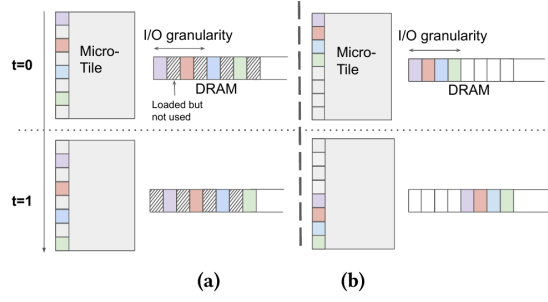


Figure 6. Uncoalesced (a) and coalesced (b) DRAM accesses. Different threads are shown in different colors.

5.2.3 Shared Memory Allocation

Figure 6: 非合并 (a) 和合并 (b) 的 DRAM 访问。不同线程用不同颜色表示。

图 6 详细解读

(a) 非合并访问：两个线程 $t = 0$ 和 $t = 1$ （不同颜色）各自访问 Micro-Tile 的一列。由于 DRAM 是按行（I/O 粒度的宽块）读取的，每次读取会带来大量「加载了但未使用」的数据（灰色区域标注）。对于一列数据，需要多次 DRAM 事务。

(b) 合并访问：编译器重新排列线程，使相邻线程访问相邻的内存地址。同一行的数据由多个线程并行读取，每次 DRAM 事务的数据都被充分利用。内存带宽利用率大幅提升。

关键启示：在 CUDA 中，程序员需要自己确保内存合并。在 Triton 中，编译器自动处理。

6.2.3 共享内存分配（Shared Memory Allocation）

原文完整翻译

具有高算术强度的 tile 级操作（如 dot）可以受益于将其操作数临时存储在快速共享内存中。共享内存分配的目的是确定何时何处应将 tile 暂存到此空间。如图 7 所示，可以通过首先计算每个感兴趣变量的存活区间（live range），然后使用 [【\[15\] Gergov 1999, 编译时内存优化算法】](#) 中提出的线性时间存储分配算法来完成。

reads from and write to shared memory are asynchronous in our machine model. The goal of the Shared Memory Synchronization pass automatically inserts *barriers* in the generated GPU source code so as to preserve program correctness. This is done by detecting read-after-writes (RAW) and write-after-read (WAR) hazards using forward data-flow analysis with the following data-flow equations:

$$\begin{aligned}
 in_s^{(RAW)} &= \bigcup_{p \in \text{pred}(s)} out_p^{(RAW)} \\
 in_s^{(WAR)} &= \bigcup_{p \in \text{pred}(s)} out_p^{(WAR)} \\
 out_s^{(RAW)} &= \begin{cases} \emptyset & \text{if } in_s^{(RAW)} \cap read(s) \neq \emptyset \text{ (barrier)} \\ \dots & \text{otherwise} \end{cases}
 \end{aligned}$$

Figure 7: 共享内存分配

图 7 详细解读

左侧——存活区间 (Live Intervals)： 每个彩色条表示一个变量在共享内存中需要存在的时间段。例如，一个 4kB 的变量在 t_1 到 t_3 期间需要留在共享内存中。

右侧——内存分配： 显示了如何将变量打包到有限的共享内存空间中。横轴是时间，纵轴是内存地址。不同变量像俄罗斯方块一样被安排在不同的地址和时间段，确保不会超过共享内存容量。

核心思想： 共享内存是 GPU 上速度最快的可编程缓存（几个时钟周期的延迟，而 DRAM 需要数百个时钟周期），但容量很小（通常 48-96 KB）。因此需要精心管理，确保在需要时数据在共享内存中，不需要时及时释放。

6.2.4 共享内存同步 (Shared Memory Synchronization)

原文完整翻译

在我们的机器模型中，对共享内存的读写是异步的。共享内存同步的目标是在生成的 GPU 源代码中自动插入屏障 (barrier)，以保证程序正确性。这通过使用前向数据流分析检测读后写 (RAW) 和写后读 (WAR) 冒险来完成，使用以下数据流方程：

$$in_s^{(RAW)} = \bigcup_{p \in pred(s)} out_p^{(RAW)} \quad (1)$$

$$in_s^{(WAR)} = \bigcup_{p \in pred(s)} out_p^{(WAR)} \quad (2)$$

$$out_s^{(RAW)} = \begin{cases} \emptyset & \text{if } in_s^{(RAW)} \cap read(s) \neq \emptyset \text{ (barrier)} \\ in_s^{(RAW)} \cup write(s) & \text{otherwise} \end{cases} \quad (3)$$

$$out_s^{(WAR)} = \begin{cases} \emptyset & \text{if } in_s^{(WAR)} \cap write(s) \neq \emptyset \text{ (barrier)} \\ in_s^{(WAR)} \cup read(s) & \text{otherwise} \end{cases} \quad (4)$$

数学推导详解

数据流方程详解

这些方程用于检测何时需要插入内存屏障 (barrier)。以 RAW (读后写) 为例：

符号含义：

- s ：当前语句 (statement)
- $pred(s)$ ： s 的所有前驱语句的集合
- $in_s^{(RAW)}$ ：进入语句 s 时，尚未被屏障保护的共享内存写操作集合
- $out_s^{(RAW)}$ ：离开语句 s 时，尚未被屏障保护的共享内存写操作集合
- $read(s)$ ：语句 s 读取的共享内存变量集合
- $write(s)$ ：语句 s 写入的共享内存变量集合

公式 (1)： $in_s^{(RAW)} = \bigcup_{p \in pred(s)} out_p^{(RAW)}$

进入 s 时的待处理写集合 = 所有前驱语句的输出写集合的并集。

公式 (3): 如果 $in_s^{(RAW)} \cap read(s) \neq \emptyset$, 说明 s 要读取的变量之前被写过但还没有屏障保护——检测到 RAW 冒险, 需要在此处插入 barrier, 清空集合。否则, 将 s 的写操作加入集合继续传播。

具体例子:

1. 语句 A: `store shared_mem, data1` (写共享内存)
2. 语句 B: `x = load shared_mem` (读共享内存)

分析: $in_B^{(RAW)} = \{shared_mem\}$, $read(B) = \{shared_mem\}$, 交集非空 $\Rightarrow$ 在 B 前插入 barrier。

WAR (写后读) 冒险的分析完全对称。

注意事项

RAW vs WAR 冒险:

- **RAW** (Read After Write) : 后续读操作可能读到旧数据 (写操作还没完成)。
- **WAR** (Write After Read) : 后续写操作可能覆盖还没读完的数据。

这两种冒险在异步共享内存操作中都可能发生, 必须通过 barrier 同步来避免。在 CUDA 中程序员需要手动插入 `__syncthreads()`, 而 Triton 自动完成。

6.3 自动调优器 (Auto-tuner)

原文完整翻译

传统自动调优器 **【[42] Van Zee & van de Geijn 2015, BLIS——快速实例化 BLAS 功能的框架】** **【[45] Whaley et al. 2000, ATLAS——软件的自动经验优化】** 通常依赖手写的参数化代码模板来在预定义工作负载上实现良好性能。相比之下, Triton-JIT 可以通过简单地连接与上述每个优化遍相关的元参数, 直接从 Triton-IR 程序中提取优化空间。

在本工作中, 只考虑了层次化分块遍, 导致每个维度每个 tile 不超过 3 个分块参数。然后使用穷举搜索在以下范围内优化这些参数: (a) tile 尺寸在 32 到 128 之间的 2 的幂次; (b) micro-tile 尺寸在 8 到 32 之间; (c) nano-tile 尺寸在 1 到 4 之间。未来可以使用更好的自动调优方法。

通俗解释

传统的自动调优器 (如 ATLAS) 需要人类专家先写好代码模板, 然后调优模板中的参数。Triton 的优势是: 编译器可以**自动**从 IR 代码中识别出可调参数, 不需要手写模板。

搜索空间的具体范围:

- Tile 尺寸: 32, 64, 128 (3 种选择)
- Micro-tile 尺寸: 8, 16, 32 (3 种选择)
- Nano-tile 尺寸: 1, 2, 4 (3 种选择)

对于一个二维问题, 每个维度有 $3 \times 3 \times 3 = 27$ 种组合, 两个维度就是 $27^2 = 729$ 种。这个

搜索空间足够小，可以用穷举搜索遍历所有组合。

7 数值实验

原文完整翻译

在本节中，我们评估 Triton 在深度学习文献中各种工作负载上的性能。我们使用 NVIDIA GeForce GTX1070，将我们的系统与最新的厂商库（cuBLAS 10.0, cuDNN 7.0）以及相关编译器技术（AutoTVM, TC, PlaidML）进行比较。在适用时，我们按照官方文档指南对这些 DSL 的每个问题规模进行了自动调优。

7.1 矩阵乘法

原文完整翻译

形如 $A = D \times W^T$ ($D \in \mathbb{R}^{M \times K}$, $W \in \mathbb{R}^{N \times K}$) 的矩阵乘法任务是神经网络计算的核心。这里我们考虑来自循环神经网络（DeepSpeech2 [【\[3\] Aodei et al. 2015, DeepSpeech2——端到端英语和普通话语音识别】](#)）和 transformer [【\[44\] Vaswani et al. 2017, Attention Is All You Need——提出 Transformer 架构】](#) 神经网络的各种任务；我们在图 8 中报告它们的性能。

Triton 和 cuBLAS 总体上性能相当，在某些任务上达到设备峰值性能的 90% 以上。然而，cuBLAS 在浅层 transformer 神经网络上仍然比 Triton 快，这要归功于使用了三维算法 [【\[2\] Agarwal et al. 1995, 三维并行矩阵乘法方法】](#)，该算法将深度归约拆分为独立块以在 M 和 N 太小时提供更多并行性。否则，现有 DSL 比我们的方案慢 2-3 倍——除了 TVM（当输入形状为 32 的倍数时不到 2 倍慢）。

various workloads from the Deep Learning literature. We used an NVIDIA GeForce GTX1070 and compared our system against the most recent vendor libraries (cuBLAS 10.0, cuDNN 7.0) as well as related compiler technology (AutoTVM, TC, PlaidML). When applicable, we auto-tuned these DSLs for each individual problem size following official documentation guidelines.

6.1 Matrix Multiplication

Matrix multiplication tasks of the form: $A = D \times W^T$ ($D \in \mathbb{R}^{M \times K}$, $W \in \mathbb{R}^{N \times K}$) are at the heart of neural network computations. Here we consider a variety of tasks from recurrent (DeepSpeech2 [3]) and transformer [44] neural networks; we report their performance in Figure 8.

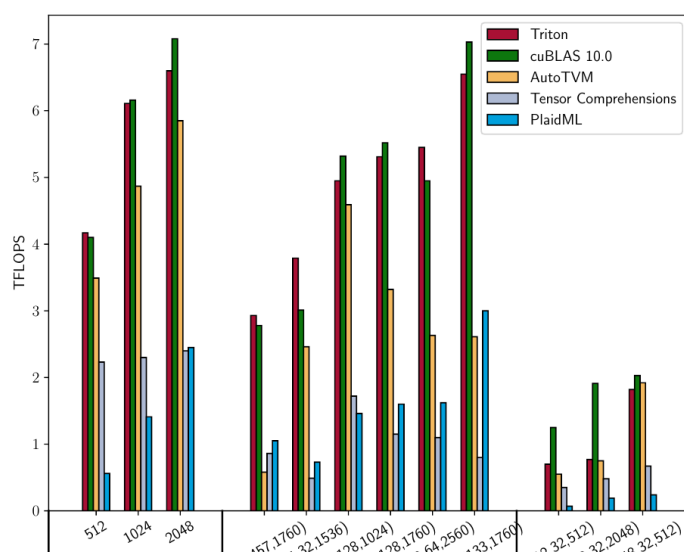


Figure 8: 矩阵乘法性能

图 8 详细解读

横轴：不同的矩阵乘法任务，分为三组：Square（方阵， $M = N = K$ ）、DeepSpeech2（语音识别模型的典型尺寸）、Transformer（注意力机制的典型尺寸）。

纵轴：性能，单位为 TFLOPS（每秒万亿次浮点运算）。GTX1070 的峰值约为 7.6 TFLOPS。

关键发现：

- **方阵**（512, 1024, 2048）：Triton（蓝色）和 cuBLAS（橙色）性能几乎一致，都接近峰值。尺寸越大越接近峰值。
- **DeepSpeech2**：大多数任务上 Triton 和 cuBLAS 相当。AutoTVM 和 PlaidML 性能明显较低。
- **Transformer**：尺寸为 (512, 32, X) 的小 M 、 N 任务中，cuBLAS 因使用 3D 算法而略胜 Triton。这是因为小的 M 、 N 意味着并行度不够，cuBLAS 的 3D 算法能沿 K 维拆分来补偿。
- **Tensor Comprehensions** 在多数任务中表现最差。

关键概念/总结

矩阵乘法性能总结：

- Triton $\approx$ cuBLAS > TVM > PlaidML > TC（一般排序）。
- Triton 达到设备峰值性能 90%+。
- 现有 DSL（TVM、TC、PlaidML）比 Triton/cuBLAS 慢 2-3 倍。
- cuBLAS 在小尺寸 Transformer 任务上因 3D 算法略有优势。

7.2 卷积

原文完整翻译

卷积神经网络（CNN）是一类重要的机器学习模型，应当被 DSL 和编译器良好支持。它们基于卷积层（图 9a），而将其实现为矩阵乘法（图 9b）对于利用专用张量处理硬件是必要的——但现有 DSL 不支持这一点。这里我们基准测试了 Triton 对 cuDNN 的“IMPLICIT_GEMM”算法的重新实现（第 6.2.1 节），并提供了移位卷积（shifted convolution）的第一个融合内核（第 6.2.2 节）。我们使用指针增量查找表来实现这些例程，如代码清单 8 所示。

ing DSLs. Here we benchmark a Triton re-implementation of cuDNN's "IMPLICIT_GEMM" algorithm (Section 6.2.1) and provide the first fused kernel available for shifted convolutions (Section 6.2.2). We implemented these routines using look-up tables of pointer increments, as shown in Listing 8.

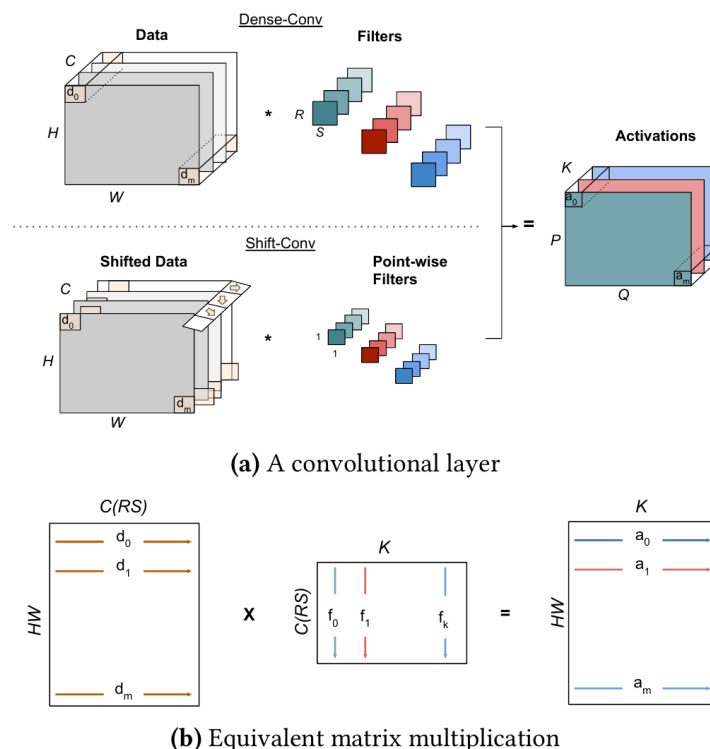


Figure 9. Dense and shifted convolutional layers (a) viewed as matrix multiplication (b)

Figure 9: 稠密卷积和移位卷积层 (a) 视为矩阵乘法 (b)

图 9 详细解读

(a) 卷积层展示了两​​种卷积方式：

Dense-Conv（稠密卷积，上方）：

- 输入数据 Data 形状为 $H \times W \times C$ （高度 $\times$ 宽度 $\times$ 通道数）。
- 滤波器 Filters 形状为 $R \times S \times C \times K$ （滤波器高 $\times$ 宽 $\times$ 输入通道 $\times$ 输出通道）。
- 输出 Activations 形状为 $P \times Q \times K$ 。
- 这是标准的卷积运算——滤波器在输入上滑动，每个位置做点积。

Shift-Conv（移位卷积，下方）：

- 先对输入数据的每个通道进行不同方向的空间移位（Shifted Data）。
- 然后使用 1×1 的逐点滤波器（Point-wise Filters）进行通道混合。

- 这是 Wu et al. 2017 提出的零 FLOP 卷积替代方案——用移位代替空间卷积。

(b) 等价矩阵乘法：卷积可以通过 `im2col` 变换转化为矩阵乘法。数据被展开为 $HW \times C(RS)$ 的矩阵（每行是一个感受野区域），滤波器被重排为 $C(RS) \times K$ 的矩阵，二者相乘得到 $HW \times K$ 的输出。

7.2.1 稠密卷积

原文完整翻译

本小节考虑的卷积层来自深度学习文献，列于表 1。

如图 10 所示，Triton 在 ResNet 上优于 cuDNN 的 IMPLICIT_GEMM 实现。这可能是因为 cuDNN 还为 3×3 卷积维护了更好的算法（即 Winograd [\[\[25\] Lavin & Gray 2016\]](#)），因此留给优化次要内核的工程资源较少。当快速算法不可用时（如 DeepSpeech2），cuDNN 和 Triton 性能相当。

Table 1: 本文考虑的卷积任务

	H	W	C	B	K	R	S	应用
Task 1	112	112	64	4	128	3	3	ResNet
Task 2	56	56	128	4	256	3	3	ResNet
Task 3	28	28	256	4	512	3	3	ResNet
Task 4	14	14	512	4	512	3	3	ResNet
Task 5	7	7	512	4	512	3	3	ResNet
Task 6	161	700	1	8	64	5	5	DeepSpeech2
Task 7	79	341	32	8	32	5	10	DeepSpeech2

表 1 解读

表中列出了 7 个卷积任务的参数：

- H, W ：输入特征图的高度和宽度
- C ：输入通道数
- B ：批大小（batch size）
- K ：输出通道数（即滤波器数量）
- R, S ：卷积核的高度和宽度
- Task 1–5 来自 ResNet [\[\[19\] He et al. 2016, ResNet——深度残差学习\]](#)，对应网络不同层的典型尺寸。随着层数加深， H, W 减小而 C, K 增大。
- Task 6–7 来自 DeepSpeech2，特征图较大（ 161×700 ），但通道数较少。

important, then fast algorithms are not available (e.g., DeepSpeech2), cuDNN and Triton are on par.

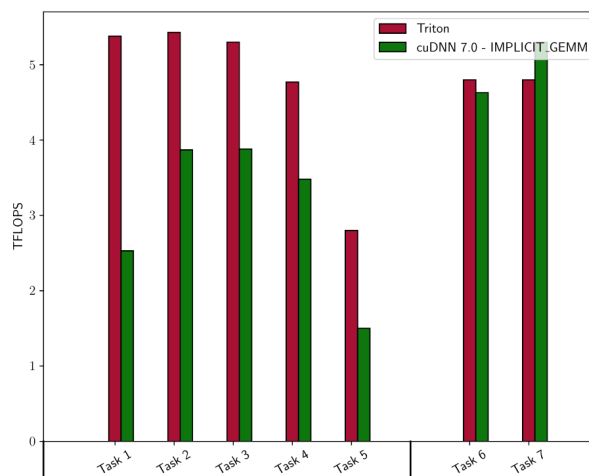


Figure 10: 隐式矩阵乘法（卷积）的性能

图 10 详细解读

横轴：7 个卷积任务（Task 1–7）。

纵轴：性能（TFLOPS）。

关键发现：

- **ResNet 任务（Task 1–5）**：Triton（蓝色）在所有任务上都优于或等于 cuDNN IMPLICIT_GEMM（橙色）。特别是 Task 2–4，Triton 有明显优势。
- **DeepSpeech2 任务（Task 6–7）**：两者性能相当。
- 论文解释 Triton 超越 cuDNN IMPLICIT_GEMM 的原因：cuDNN 团队把更多工程资源投入到 Winograd 等更优算法上，对 IMPLICIT_GEMM 路径的优化投入较少。

7.2.2 移位卷积（Shift Convolutions）

原文完整翻译

我们最后考虑将表 1 中 Task 1–5 作为移位卷积来实现——这是 CNN 的一种新方法（参见图 9a）。我们将 Triton 中融合 shift-conv 模块的实现（代码清单 8）与一种朴素实现进行比较，后者依赖手写的移位内核和单独调用 cuBLAS。我们还报告了不做移位时（即 1×1 卷积）的最大可达性能。如图 11 所示，我们的 Triton 实现几乎能完全隐藏移位的开销。

代码清单 8：Triton-C 中的移位卷积

Listing 8: Triton-C 中的移位卷积实现

```
1 const tunable int TM = {16, 32, 64, 128};
2 const tunable int TN = {16, 32, 64, 128};
3 const tunable int TK = {8};
4
5 __constant__ int* delta = alloc_const int[512];
```

```

6
7 for(int c = 0; c < C; c++)
8     delta[c] = c*H*W + shift_h[c]*W + shift_w[c]
9
10 void shift_conv(restrict read_only float *a,
11               restrict read_only float *b, float *c,
12               int M, int N, int K){
13     int rxa[TM] = get_global_range[TM](0);
14     int ryb[TN] = get_global_range[TN](1);
15     int rka[TK] = 0 ... TK;
16     int rkb[TK] = 0 ... TK;
17     float C[TM, TN] = 0;
18     float* pxa[TM, TK] = a + rxa[:, newaxis];
19     float* pb[TN, TK] = b + ryb[:, newaxis] + rkb*N;
20     __constant__ int* pd[TK] = delta + rka;
21     for(int k = K; k > 0; k = k - TK){
22         int delta[TK] = *pd;
23         float* pa[TM, TK] = pxa + delta[newaxis, :];
24         float a[TM, TK] = *pa;
25         float b[TN, TK] = *pb;
26         C = dot(a, trans(b), C);
27         pb = pb + TK*N;
28         pd = pd + TK;
29     }
30     int rxc[TM] = get_global_range[TM](0);
31     int ryc[TN] = get_global_range[TN](1);
32     float* pc[TM, TN] = c + rxc[:, newaxis] + ryc*M;
33     bool checkc0[TM] = rxc < M;
34     bool checkc1[TN] = ryc < N;
35     bool checkc[TM, TN] = checkc0[:, newaxis] && checkc1;
36     @checkc *pc = C;
37 }

```

代码清单 8 逐段解读

这段代码实现了**融合的移位卷积**，核心技巧是使用指针增量查找表 `delta`。

第5–8行：预计算 `delta` 查找表。对每个输入通道 c ，计算移位后的内存偏移量： $\text{delta}[c] = c \cdot H \cdot W + \text{shift_h}[c] \cdot W + \text{shift_w}[c]$ 。这编码了每个通道独特的空间移位方向和距离。

第21–22行：在循环内部，从查找表读取当前 TK 个通道的偏移量，然后计算实际的数据指针。这就是「融合」的精髓——移位操作被编码为指针偏移，不需要单独的移位内核。

第26行：`dot(a, trans(b), C)` 执行矩阵乘法并累加，等同于 $C += a \times b^T$ 。

关键创新：传统做法需要两步——先移位（单独的内核），再做 1×1 卷积（cuBLAS）。Triton 将两步融合为一步，通过查找表实现动态指针偏移，避免了额外的内存读写。

```

int rxc[TM] = get_global_range[TM](0);
int ryc[TN] = get_global_range[TN](1);
float* pc[TM, TN] = c + rxc[:, newaxis] + ryc*M;
bool checkc0[TM] = rxc < M;
bool checkc1[TN] = ryc < N;
bool checkc[TM, TN] = checkc0[:, newaxis] && checkc1;
@checkc *pc = C;
}

```

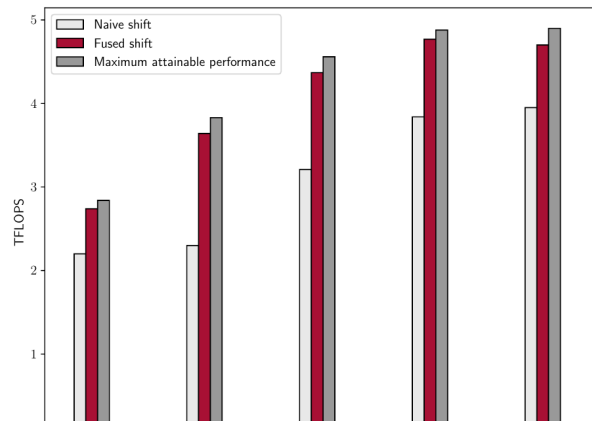


Figure 11: Triton 中移位卷积的性能

图 11 详细解读

横轴：Task 1–5（ResNet 的 5 个卷积层作为移位卷积实现）。

纵轴：性能（TFLOPS）。

三种比较：

- **Naive shift**（紫色）：朴素实现（先手写移位内核 + 再调用 cuBLAS）。性能最低。
- **Fused shift**（蓝色）：Triton 融合实现。性能大幅提升。
- **Maximum attainable**（绿色虚线）：理论上限（ 1×1 卷积，不做移位）。

关键结论：Triton 的融合实现几乎达到了不做移位时的性能上限，说明移位操作的开销被成功隐藏。而朴素实现因为额外的内核启动开销和数据搬运，性能损失明显。这展示了 Triton 在实现新型计算模式时的优势。

8 结论

原文完整翻译

在本文中，我们提出了 Triton，一种用于表达和编译分块神经网络计算为高效机器代码的开源语言和编译器。我们展示了仅向 LLVM-IR 添加少量数据流和控制流扩展就能启用各种 tile 级优化遍，这些优化遍共同实现了与厂商库相当的性能。我们还提出了 Triton-C，一种更高级的语言，在其中我们能够简洁地实现面向 CNN 新型神经网络架构的高效内核。

未来工作方向包括对 Tensor Core 的支持、量化内核 [【\[26\] Lin et al. 2016, 深度卷积网络的定点量化——量化方法】](#) 的实现，以及集成到更高级的 DSL 中。

通俗解释

论文总结了 Triton 的核心贡献：通过在编译器 IR 层面引入 tile 级操作和优化，实现了与手工优化库相当的性能，同时大幅降低了编程复杂度。

未来方向值得注意：

- **Tensor Core 支持**：NVIDIA 的 Tensor Core 是专门用于矩阵乘法的硬件单元，支持后性能会进一步提升。（注：后续版本的 Triton 已经实现了这一目标。）
- **量化内核**：使用低精度（如 INT8）计算以提高吞吐量。
- **集成到更高级 DSL**：让 Triton 成为 TVM 等框架的后端。

全文核心要点总结

1. **核心创新**：以 tile 为第一等公民的编译器 IR 和优化体系。
2. **三层架构**：Triton-C（前端语言）→ Triton-IR（中间表示）→ Triton-JIT（即时编译器）。
3. **关键优化**：层次化分块、内存合并、共享内存管理、自动同步、预取。
4. **性能**：矩阵乘法和卷积均可媲美 cuBLAS/cuDNN，远超其他 DSL。
5. **灵活性**：能高效实现新型操作（如移位卷积），这是纯库方案无法做到的。
6. **编程模型简化**：单线程 tile 编程，无需手动管理线程同步。

9 致谢

原文完整翻译

本工作得到了 NVIDIA 研究生奖学金的支持。

附录：参考文献一览表

以下表格列出了论文中所有引用的文献，包括其主要内容和在本文中被引用的原因。

文献	主要内容	在本文中的引用原因
[1] Abadi et al. 2016, TensorFlow	Google 的大规模机器学习系统	代表性深度学习框架
[2] Agarwal et al. 1995, 3D GEMM	三维并行矩阵乘法方法	解释 cuBLAS 在小尺寸任务上的优势
[3] Amodei et al. 2015, DeepSpeech2	端到端语音识别系统	提供矩阵乘法和卷积的测试任务
[4] Auguin & Larbey, Opsila	SIMD 数值分析处理器	SPMD 编程模型的参考
[5] Bao & Ding 2013	防御性循环分块（共享缓存）	分析性分块启发式的例子
[6] Baskaran et al. 2010	参数化分块方法	经验性分块启发式的例子
[7] Braun et al. 2013	SSA 形式的简单高效构造	Triton-IR 使用 SSA 形式的理论基础
[8] Carter et al. 1999, PSSA	谓词化静态单赋值	Triton-IR 的谓词化控制流机制
[9] Chen et al. 2015, MXNet	灵活高效的分布式 ML 库	代表性深度学习框架
[10] Chen et al. 2018, TVM	端到端深度学习优化栈	主要竞争对手和对比对象
[11] Chen et al. 2018, TVM (dup)	同 [10]	微内核和参数化调度的参考
[12] Davidson & Jinturkar 1994	内存访问合并技术	tile 语义隐藏的性能细节之一
[13] Diamos et al. 2016	Persistent RNNs	需要专家手写 GPU 内核的例子
[14] Elango et al. 2018, Diesel	GPU 线性代数/NN 的 DSL	多面体模型方法的代表
[15] Gergov 1999	编译时内存优化算法	共享内存分配算法的来源
[16] Google 2017, XLA	TensorFlow 加速线性代数	张量级 IR 方法的代表
[17] Gray et al. 2017	块稀疏权重的 GPU 内核	需要专家手写内核的例子
[18] Griehl et al. 1998	多面体模型中的代码生成	多面体编译方法的理论基础
[19] He et al. 2016, ResNet	深度残差学习	卷积基准测试的来源
[20] Howard et al. 2017, MobileNets	轻量级 CNN 架构	DSL 在深度可分离卷积上表现尚可
[21] Huang & van de Geijn 2016	BLISlab GEMM 优化沙盒	微内核方法的代表
[22] Intel AI 2018, PlaidML	Intel 的开源 DL 编译器	主要竞争对手和对比对象

文献	主要内容	在本文中的引用原因
[23] Kerr et al., CUT-LASS	CUDA C++ 线性代数库	高级分块编程抽象的例子
[24] Krizhevsky et al. 2012	ImageNet / AlexNet	GPU 加速 DL 时代的开始
[25] Lavin & Gray 2016	Winograd 快速卷积	专家优化内核的例子; cuDNN 的替代算法
[26] Lin et al. 2016	深度 CNN 的定点量化	未来工作方向之一
[27] Martineau et al. 2017	V100/Tensor Core 基准测试	tile 语义隐藏的硬件利用细节
[28] McDanel et al. 2017	嵌入式二值化 NN	结构化稀疏模式的例子
[29] McKeeman 1965	窥孔优化	Triton tile 级窥孔优化的参考
[30] Mehta et al. 2016, TurboTiling	利用预取提升分块性能	经验性分块启发式的例子
[31] Narang et al. 2017	块稀疏 RNN	结构化稀疏模式的例子
[32] Nath et al. 2011	加速 GPU 稠密线性代数	tile 语义隐藏的缓存管理细节
[33] Nickolls et al. 2008, CUDA	可扩展的并行编程	CUDA 编程模型的参考
[34] Nugteren 2017, CLBlast	调优的 OpenCL BLAS 库	自动调优框架中分块技术的例子
[35] Oliphant 2006, NumPy	Python 数值计算库	Triton 广播语义的来源
[36] PyTorch 2016	开源深度学习框架	代表性深度学习框架
[37] Ragan-Kelley et al. 2013, Halide	图像处理语言和编译器	循环合成方法的代表
[38] Rotem et al. 2018, Glow	神经网络图降低编译器	张量级 IR 方法的代表
[39] Stoutchinin & de Ferriere 2001	谓词化的高效 SSA 形式	ψ -函数的理论来源
[40] Tillet & Cox 2017	输入感知的自动调优	自动调优框架中分块技术的例子
[41] Unat et al. 2016, TiDA	数据局部性管理编程抽象	高级分块编程抽象的例子
[42] Van Zee & van de Geijn 2015, BLIS	快速实例化 BLAS 的框架	传统自动调优方法的代表
[43] Vasilache et al. 2018, TC	框架无关的高性能 ML 抽象	多面体方法代表; 主要竞争对手
[44] Vaswani et al. 2017, Transformer	Attention Is All You Need	提供矩阵乘法测试任务
[45] Whaley et al. 2000, ATLAS	软件的自动经验优化	传统自动调优方法的代表
[46] Williams et al. 2009, Roofline	多核架构的性能模型	图 1 中使用的 Roofline 模型

文献	主要内容	在本文中的引用原因
[47] Wu et al. 2017, Shift	零 FLOP 空间卷积替代方案	移位卷积实验的理论来源
[48] Zhao et al. 2018	数据中心循环分块	分析性分块启发式的例子